PROJECT REPORT

Arbitrary Precision Calculator (APC)

Developed in: C Programming Language

Architecture: Command-Line Interface (CLI)

Core Data Structure: Doubly Linked Lists (DLL)

1. INTRODUCTION

In standard C programming, primitive data types are strictly bound by hardware architecture limits. For instance, a 64-bit unsigned integer maximum capacity is approximately 18.4 quintillion. Applications requiring cryptographic key generation, high-precision scientific calculations, or massive factorials will instantly trigger integer overflow. The Arbitrary Precision Calculator (APC) is a custom software utility designed to bypass hardware limits. By representing numbers as dynamic Doubly Linked Lists in heap memory, it can execute arithmetic operations on numbers of theoretically infinite magnitude.

2. SYSTEM ARCHITECTURE & MODULES

The APC is compartmentalized into functional operational files, separating the user interface and sign-routing logic from the raw mathematical algorithms.

2.1. Routing & Initialization (main.c)

This module captures command-line arguments in the format `operand1 operator operand2`. It extracts the numerical strings, identifies trailing negative signs, and parses the strings character-by-character into two distinct Doubly Linked Lists. A complex routing switch-statement intercepts negative combinations (e.g., adding a negative and a positive) and redirects them to the appropriate absolute-value arithmetic functions, applying the correct sign to the final output.

2.2. Arithmetic Modules (addition.c, subtraction.c, etc.)

The mathematical operations mimic manual, grade-school arithmetic. Because numbers are calculated starting from the least significant digit, the algorithms traverse the Doubly Linked Lists backwards—from the Tail pointer to the Head pointer—calculating sums or differences and passing carry/borrow values to the subsequent nodes.

3. DATA STRUCTURES & CORE LOGIC

The structural backbone of the APC is the Doubly Linked List, allowing bidirectional traversal which is critical for alignment and carry operations.

Node Structure

```
typedef struct node {
    struct node *prev;
    int data;          // Holds a single digit (0-9)
    struct node *next;
} Dlist;
```

Optimized Magnitude Comparison

Before subtraction, the program must identify the larger number to prevent negative intermediate results. The `list_compare` function is highly optimized: it first calculates the total node count of both lists. If List A has more nodes than List B, it is instantly recognized as greater—bypassing the need for an expensive $O(N)$ digit-by-digit comparison unless the lengths are exactly equal.

4. KEY FEATURES & EDGE CASES

Feature	Implementation Details
Multiplication Shift Logic	Mimics long multiplication using nested loops. The outer loop traverses the bottom number. The inner loop multiplies against the top number. Crucially, a counter injects trailing zeros (`0` nodes) to simulate the tens/hundreds place shift before accumulating the sub-totals.
Leading Zero Cleanup	If subtracting 1000 from 1001, the raw algorithmic output might be `0001`. A dedicated loop traverses the head of the resultant list and frees leading zero nodes until it hits a non-zero digit.
Sign Agnosticism	The core math functions (`addition.c`, `subtraction.c`) are completely agnostic to positive/negative signs. They strictly compute absolute values, while `main.c` handles the sign assignment, keeping the logic cleanly separated.

5. LIMITATIONS AND FUTURE ENHANCEMENTS

While the APC successfully achieves arbitrary precision, it provides an excellent foundation for significant algorithmic scaling:

- **Memory Compression (Base 1 Billion):** Currently, each node stores a single digit (0-9), meaning a 24-byte node is used to store 4 bits of data. A future enhancement involves compressing up to 9 digits (e.g., `999,999,999`) into a single integer node. This "Base 10^9" approach drastically reduces heap allocation and speeds up traversal operations by a factor of nine.
- **Algorithmic Efficiency in Division:** The current `division.c` module utilizes a "repeated subtraction" method. For calculations like $1,000,000 / 1$, the loop executes 1 million times, resulting in $O(Q)$ time complexity. Implementing a Binary Search Division or Long Division algorithm would reduce this complexity to $O(N)$.
- **Memory Leak Mitigation:** The program dynamically allocates nodes for intermediate totals during multiplication. While some cleanup exists, enforcing a rigorous post-order traversal function to `free()` all DLL nodes before program termination is necessary to guarantee absolute memory safety for daemonized instances.

6. CONCLUSION

The Arbitrary Precision Calculator project is a rigorous exercise in algorithmic design and dynamic memory management. By successfully escaping hardware-bound integer limits using custom Data Structures, the project demonstrates advanced proficiency in C programming, pointer mechanics, and the practical implementation of mathematical algorithms at the systems level.